

Penentuan Arah

Pegunungan Bukit Barisan di Pulau Sumatra memiliki banyak gunung tinggi, termasuk lebih dari 30 puncak dengan ketinggian di atas 1.000 meter. Aming berencana membangun jaringan kabel luncur robotik di Pegunungan Bukit Barisan untuk mengirim paket di antara desa-desa terpencil di sana.

Dalam rencana Aming, jaringan tersebut terdiri atas N titik, dinomori dari 0 sampai $N - 1$. Di antara titik-titik ini, **tepat** $K = 6$ di antaranya merupakan tempat pengisian daya, sementara $N - K$ titik sisanya merupakan desa. Titik-titik ini dihubungkan oleh $N - 1$ kabel **dua arah**, dinomori dari 0 sampai $N - 2$. Untuk setiap i ($0 \leq i \leq N - 2$), kabel i menghubungkan titik $U[i]$ dan $V[i]$. Setiap kabel menghubungkan dua titik yang berbeda, dan setiap pasang titik dihubungkan oleh paling banyak satu kabel. Dijamin bahwa dari setiap titik, semua titik lainnya dapat dicapai menggunakan kabel-kabel yang ada.

Jarak titik a dan b ditulis $d(a, b)$ dan didefinisikan sebagai berikut.

- Jika $a = b$, $d(a, b) = 0$.
- Selebihnya, $d(a, b)$ adalah banyaknya kabel minimum yang dibutuhkan untuk berpindah dari a ke b .

Kita katakan bahwa jaringan ini memiliki **faktor percabangan** setidaknya δ jika setiap titik pada jaringan terhubung dengan tepat 1 kabel atau setidaknya δ kabel. Aming mengetahui suatu bilangan bulat B sehingga faktor percabangan dari jaringan ini setidaknya B .

Aming menyiapkan 100 jenis kabel berbeda, dinomori $0, 1, \dots, 99$. Setiap kabel pada jaringan akan berupa salah satu dari jenis-jenis tersebut.

Sejumlah robot akan menyusuri kabel-kabel luncur ini untuk melakukan pengiriman. Aming ingin memasang perangkat lunak yang membantu para robot kembali ke tempat pengisian daya dan mengisi daya mereka. Akan tetapi, robot-robot ini memiliki kapasitas memori yang sangat terbatas. Maka, dalam mendesain perangkat lunak penentuan arah, robot-robot ini hanya boleh menentukan arah berdasarkan banyaknya tempat pengisian daya $K = 6$, faktor percabangan terjamin B , serta jenis-jenis kabel yang terhubung pada posisi robot saat ini.

Ketika suatu robot ingin menentukan arah pada titik desa u yang terhubung dengan dua kabel atau lebih, robot tersebut pertama-tama memindai kabel-kabel yang terhubung dengan u dalam urutan sembarang. Perangkat lunaknya akan diberikan daftar jenis-jenis kabel dalam urutan

tersebut. Untuk setiap kabel, perangkat lunak ini harus menghitung banyaknya tempat pengisian daya yang jaraknya dari robot akan berkurang jika robot menyusuri kabel tersebut.

Secara formal, misalkan $v_1, \dots, v_k$ ($k \geq 2$) merupakan titik-titik yang terhubung langsung dengan u melalui kabel, dan c_i ($1 \leq i \leq k$) merupakan jenis kabel yang menghubungkan titik u dan v_i . Perangkat lunak ini akan diberikan nilai K , B , dan daftar $c_1, c_2, \dots, c_k$. Untuk setiap i ($1 \leq i \leq k$), perangkat lunak ini harus menentukan banyaknya tempat pengisian daya p yang memenuhi $d(v_i, p) < d(u, p)$.

Tugas Anda adalah menyusun strategi penentuan jenis kabel dan merancang perangkat lunak penentuan arah. Nilai solusi Anda bergantung pada banyaknya jenis kabel berbeda yang digunakan (lihat bagian Subsoal dan Penilaian untuk rinciannya).

Detail Implementasi

Anda harus mengimplementasikan dua prosedur, satu untuk menentukan jenis-jenis kabel, dan satu lagi untuk perangkat lunak robot.

Prosedur yang harus Anda implementasikan untuk **menentukan jenis-jenis kabel** adalah:

```
std::vector<int> construct_network(int N, int K, int B,
                                  std::vector<int> U, std::vector<int> V,
                                  std::vector<int> P)
```

- N : banyaknya titik.
- K : banyaknya tempat pengisian daya.
- B : faktor percabangan minimum yang dijamin pada jaringan.
- U, V : array sepanjang $N - 1$ yang menjelaskan kabel-kabel pada jaringan.
- P : array sepanjang $K = 6$ yang berisi titik-titik tempat pengisian daya.
- Prosedur ini dipanggil **paling banyak 10 000 kali** untuk setiap kasus uji.

Prosedur ini harus mengembalikan suatu array T dengan panjang $N - 1$:

- Sebuah kabel dengan jenis $T[i]$ digunakan untuk menghubungkan titik $U[i]$ dan $V[i]$.
- Setiap nilai $T[i]$ harus memenuhi $0 \leq T[i] < 100$.

Prosedur yang harus Anda implementasikan untuk **perangkat lunak robot** adalah:

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K : banyaknya tempat pengisian daya.
- B : faktor percabangan minimum yang dijamin pada jaringan.

- C : daftar jenis kabel yang terhubung ke **titik-titik desa yang terhubung dengan setidaknya 2 kabel** dalam urutan sembarang.
- Panjang C dijamin setidaknya B .
- Prosedur ini dipanggil paling banyak 100 000 kali untuk setiap kasus uji.
- Prosedur ini tidak boleh bergantung pada struktur jaringan awal; khususnya, sistem *grading* dapat mengatur ulang urutan pemanggilan prosedur ini di antara jaringan-jaringan yang berbeda dalam satu kasus uji.

Prosedur ini harus mengembalikan suatu *array* D :

- Panjang *array* D harus sama dengan panjang *array* C .
- $D[i]$ adalah banyaknya tempat pengisian daya yang jaraknya dari robot akan berkurang jika robot menyusuri kabel yang bersesuaian dengan $C[i]$.

Perhatikan bahwa pada sistem *grading*, prosedur `construct_network` dan `navigate` dipanggil **pada dua proses berbeda**.

Batasan

- $K = 6$.
- $K < N \leq 100\,000$.
- Jumlah N dari semua pemanggilan `construct_network` tidak melebihi 100 000 dalam setiap kasus uji.
- $0 \leq U[i] < V[i] < N$ untuk setiap $0 \leq i \leq N - 2$.
- Setiap titik dapat mencapai semua titik lainnya menggunakan kabel-kabel yang ada.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- Masing-masing *array* C merupakan permutasi dari daftar jenis kabel yang terhubung ke titik-titik desa yang terhubung dengan setidaknya 2 kabel.
- Jumlah panjang *array* C dari semua pemanggilan `navigate` tidak melebihi 200 000 dalam setiap kasus uji.

Subsoal dan Penilaian

Subsoal	Nilai	Batasan Tambahan
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ untuk setiap i sehingga $0 \leq i < N - 1$.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

Pada kasus uji mana pun, jika setidaknya satu dari syarat-syarat berikut terpenuhi, nilai solusi Anda pada kasus uji tersebut adalah 0 (dilaporkan sebagai `Output isn't correct` pada CMS):

- Nilai kembalian pemanggilan `construct_network` mana pun tidak valid.
- Nilai kembalian pemanggilan `navigate` mana pun tidak valid.

Selebihnya, misalkan S adalah bilangan bulat terkecil yang lebih besar dari semua elemen dalam setiap *array* T yang dikembalikan oleh pemanggilan-pemanggilan `construct_network`. Dengan kata lain, S adalah bilangan bulat terkecil sehingga semua jaringan yang dibuat hanya menggunakan jenis kabel $0, 1, \dots, S - 1$.

Nilai Anda untuk masing-masing subsoal bergantung pada nilai S sebagai berikut:

Ketentuan	Subsoal 1	Subsoal 2	Subsoal 3 dan 4	Subsoal 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

Khususnya, pada Subsoal 1, 2, dan 5, Anda akan menerima nilai penuh jika $S \leq 5$, dan pada Subsoal 3 dan 4, Anda akan menerima nilai penuh jika $S \leq 4$.

Contoh

Perhatikan skenario dengan $N = 9$, $K = 6$, dan $B = 2$.

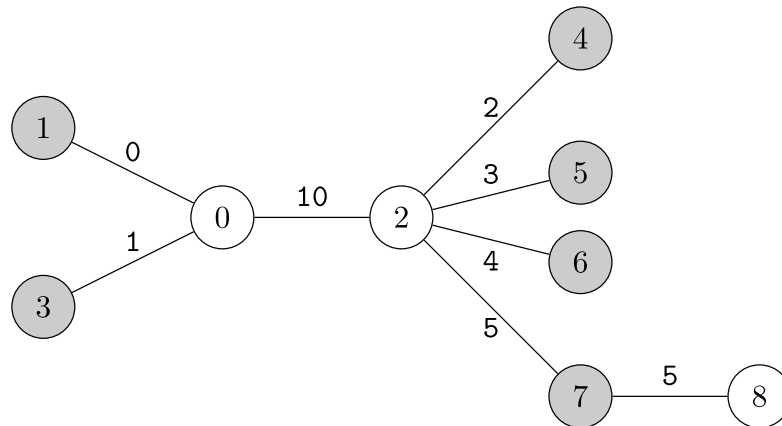
Struktur jaringan dideskripsikan oleh $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$. Tempat pengisian daya terletak pada $P = [1, 3, 4, 5, 6, 7]$. Perhatikan pemanggilan berikut pada proses pertama:

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

Misalkan Aming membuat jaringan tersebut dengan mengembalikan:

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

Jaringan yang dibuat dapat digambarkan sebagai berikut.



Kemudian, pada proses kedua, misalkan suatu robot perlu menentukan arah dari titik 0. Titik 0 terhubung dengan titik 1, 2, dan 3. Jika robot tersebut membaca jenis kabel dalam urutan tersebut, akan dilakukan pemanggilan berikut:

```
navigate(6, 2, [0, 10, 1])
```

Berdasarkan struktur jaringan:

- Tempat pengisian daya pertama, yakni titik 1, memiliki jarak terdekat ke dirinya sendiri. Khususnya, $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$.
- Tempat pengisian daya kedua, yakni titik 3, memiliki jarak terdekat ke dirinya sendiri. Khususnya, $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$.
- Tempat pengisian daya lainnya, yakni titik 4, 5, 6, dan 7, memiliki jarak yang lebih dekat ke titik 2. Khususnya, $1 = d(2, u) < 2 = d(0, u) < 3 = d(1, u) = d(3, u)$ ketika $u = 4, 5, 6$, atau 7.

Banyaknya tempat pengisian daya yang jaraknya berkurang setelah menyusuri kabel $(0, 1)$, $(0, 2)$, dan $(0, 3)$ adalah 1, 4, dan 1 secara berturut-turut. Jadi, prosedur ini harus mengembalikan:

```
[1, 4, 1]
```

Perhatikan bahwa `navigate` bisa saja dipanggil untuk permutasi-permutasi lain dari C . Sebagai contoh, `navigate(6, 2, [1, 0, 10])` juga merupakan pemanggilan yang valid untuk titik 0. Pada kasus tersebut, prosedur itu harus mengembalikan `[1, 1, 4]`.

Misalkan suatu robot perlu menentukan arah dari titik 2. Akan dilakukan pemanggilan berikut:

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

Prosedur ini harus mengembalikan:

```
[2, 1, 1, 1, 1]
```

Pada jaringan ini, prosedur `navigate` **tidak mungkin** dipanggil pada titik lain, karena:

- titik 1, 3, 4, 5, 6, dan 7 adalah tempat pengisian daya, dan
- titik 8 hanya terhubung dengan satu kabel.

Pada kasus uji ini, jenis kabel yang digunakan adalah 0, 1, 2, 3, 4, 5, dan 10. Maka, $S = 11$ akan digunakan dalam penentuan nilai.

Selain contoh di atas, paket lampiran soal ini berisi dua contoh masukan dan keluaran lain untuk contoh *grader*.

Contoh Grader

Contoh *grader* memanggil `construct_network` dan `navigate` pada proses yang sama. Selain itu, ketika contoh *grader* memanggil `navigate`, urutan jenis kabel akan bersesuaian dengan urutan kemunculan kabel pada masukan. Kedua perilaku ini berbeda dari *grader* sesungguhnya.

Format masukan:

```
T
(skenario 0)
(skenario 1)
...
(skenario T-1)
```

Masing-masing skenario diberikan dalam format berikut:

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

Pada masing-masing skenario, contoh *grader* akan memanggil `navigate` sebanyak Q kali; pemanggilan ke- i menggunakan informasi titik $X[i - 1]$. Perhatikan bahwa contoh *grader* juga mendukung nilai K berupa bilangan bulat selain 6.

Format keluaran:

Jika *array* kembalian mana pun dari `construct_network` atau `navigate` tidak valid, contoh *grader* akan menghentikan eksekusi dan mengeluarkan alasannya. Selebihnya, contoh *grader* akan mengeluarkan masing-masing *array D* dari pemanggilan `navigate` dalam satu baris.

Setelah semua skenario selesai dijalankan, contoh *grader* mengeluarkan satu baris ke *standard error*:

```
S = S'
```

dengan S' merupakan bilangan bulat terkecil yang lebih besar dari semua elemen dalam setiap *array T* yang dikembalikan oleh pemanggilan-pemanggilan `construct_network` pada kasus uji ini.